Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»

Кафедра вычислительных систем

ОТЧЕТ
по практической работе 8

по дисциплине **«Основы разработки на Java»**

Выполнил:
студент гр. ИС-542
«1» ноября 2026 г. _____ /Горбачев Д.В./

Проверил:
Ассистент кафедры ВС
«1» ноября 2026 г. _____ /Гузенков С.К./

Оценка «_____»

Новосибирск 2026

Ссылка на репозиторий: https://github.com/Mahmud52Ali/java/tree/feat/pr_08

В работе реализованы детерминированная сортировка каталога курсов, flatMap и reduce, группировка через Collectors, поиск Optional-результата, собственный collector и однопроходный отчёт CourseHub.

Задание 1 Сортировка и детерминизм

Отсортировать курсы по уровню, длительности и коду с явными критериями для равных значений. Для одинакового набора данных результат не должен зависеть от исходного порядка.

Task01DeterministicSorting задаёт comparator по sortOrder уровня, длительности, CourseCode и ID. Один и тот же каталог сортируется в исходном и обратном порядке.

Обе сортировки дали порядок SQL-101, JAVA-101, JAVA-STREAMS, JAVA-102, WEB-101. Списки совпали.

Задание 2 flatMap и обратный индекс тем

Из модулей курсов получить уникальные нормализованные темы и индекс тема → курсы, не собирая вручную вложенный список.

Task02FlatMapTopics разворачивает модули через flatMap, нормализует текст и группирует коды курсов в TreeSet по теме. TreeMap задаёт стабильный порядок тем.

Повторяющиеся варианты «ООП» объединены в одну тему, связанную с JAVA-101, JAVA-102 и JAVA-STREAMS. Тема sql связана с SQL-101 и WEB-101.

Задание 3 reduce и identity

Посчитать сумму длительностей и объединить value objects. Показать неправильную identity и объяснить ассоциативность accumulator.

Длительность задана неизменяемым CourseDuration. Его plus создаёт новый объект с суммой часов. reduce запускается с CourseDuration.ZERO; отдельно показан запуск с ошибочной identity, равной одному часу.

Сумма пяти курсов равна 198 ч, а неверная identity даёт 199 ч. Для пустого потока с ней результат тоже равен 1 ч. plus ассоциативен: (a + b) + c = a + (b + c).

Задание 4 Группировка с Collectors

Сгруппировать курсы по уровню, вычислить количество и среднюю длительность и собрать summary object без трёх полных проходов по каталогу.

Task04CollectorsSummary использует один groupingBy с summarizingLong. Он за один проход собирает count и сумму часов, из которых вычисляется среднее; результат помещается в CourseCatalogSummary.

Начальный уровень: 2 курса и 27 ч в среднем; средний: 2 курса и 42 ч; продвинутый: 1 курс и 60 ч. Отдельные проходы для count, суммы и средней не выполняются.

Задание 5 Optional как результат

Найти самый длинный курс, преобразовать найденное значение и задать fallback. Не использовать Optional.get без доказанного наличия и не хранить Optional в поле сущности.

Task05OptionalResult получает максимум по длительности, преобразует Course в строку через map и задаёт резервный текст через orElse. Optional используется только как результат операции.

Для каталога выбран WEB-101. Для пустого каталога выводится «Каталог курсов пуст»; Optional.get не вызывается.

Задание 6 Собственный collector

Создать статистику count/sum/min/max без mutable static state. Проверить combiner на разбиении и вернуть неизменяемый результат; сравнить с готовым summarizing collector.

Task06CustomCollector использует отдельный Accumulator с count, sum, min и max. Combiner объединяет две частичные статистики; parallelStream проверяет его работу. Finisher возвращает CourseStatistics — неизменяемую record-модель.

Результат собственного collector совпал с summarizingLong: count 5, sum 198, минимум 24 и максимум 60. Общего изменяемого состояния нет.

Задание 7 Однопроходный отчёт CourseHub

За один проход событий получить уникальных студентов, completion rate по курсам, top-3, распределение по уровню и аномалии. Результат детерминирован, примеры ошибок ограничены, внешних side effects нет.

CourseHubAnalyzer принимает одноразовый Stream и обрабатывает его циклом, одновременно обновляя локальные счётчики и множества. Неизвестный курс, неположительный ID студента и повторная запись считаются аномалиями. Top-3 сортируется по числу уникальных записей, при равенстве — по коду. Сохраняются не более 10 примеров, но общий счёт и распределение аномалий считаются полностью.

В демонстрации учтены 7 положительных ID студентов, показаны completion rate по пяти курсам, top-3 и распределение 2/2/1 по уровням. Найдены три аномалии — по одной каждого типа. Точный подсчёт уникальности требует памяти O(U + R) для ID студентов и пар «студент—курс»; остальные сводки занимают O(C), а примеры аномалий — не более 10 записей. Для нескольких связанных агрегатов обычный цикл яснее цепочки Stream-операций; flatMap и Collectors используются в отдельных преобразованиях каталога.

Команда gradle clean run успешно собрала проект и выполнила демонстрации всех семи заданий.